

AI-ASSISTED TRAVEL PLANNER

Section A: Literature Review

A.1 Introduction

This section reviews existing travel planning platforms and systems, relevant artificial intelligence and recommendation technologies, and academic research informing the design of the AI-Assisted Travel Planner. The review establishes the theoretical foundation for the system and identifies the research gap addressed by this project.

A.2 Existing Travel Planning Systems

Several commercial platforms currently serve the travel planning market. An analysis of these systems reveals a common pattern: strong transactional features paired with limited support for the exploratory, preference-driven phase of trip planning.

A.2.1 Google Travel

Google Travel aggregates flight search, hotel listings, and trip organisation tools within a single interface backed by Google's search and Maps infrastructure. While comprehensive in scope, the platform primarily surfaces options based on search queries rather than personalised preference profiling. There is no conversational or interactive intelligence layer for destination discovery; recommendations are generated algorithmically from search history and browsing behaviour rather than from explicit, structured user preferences. The system is optimised for users who already know where they wish to travel, offering limited support for open-ended destination exploration.

A.2.2 TripAdvisor

TripAdvisor provides one of the largest repositories of user-generated travel content, including reviews of hotels, restaurants, and attractions. Its intelligent features are primarily review-summarisation and personalised content ranking based on historical interactions. The platform is destination-centric and does not offer an integrated trip planning lifecycle — users cannot manage a trip from ideation through to history tracking within a single coherent workflow. Furthermore, its booking integrations redirect users to third-party providers, fragmenting the planning experience.

A.2.3 Expedia and Booking.com

Expedia and Booking.com represent the dominant commercial booking platforms. Both focus on transactional flows: flight and accommodation search, price comparison, and payment processing. Neither platform provides destination recommendation based on structured preference input, nor do they support a manual planning mode in which users curate their own itinerary without immediately committing to a booking. Their complexity and commercial orientation make them unsuitable as lightweight planning tools for users in the early ideation phase of trip planning.

A.2.4 Roadtrippers and Wanderlog

Roadtrippers and Wanderlog are itinerary-focused tools that allow users to map and schedule multi-stop trips. Wanderlog, in particular, supports automated itinerary suggestions. However, these tools do not integrate flight search, and their automated features produce day-by-day itineraries rather than destination recommendations based on user preference profiles such as climate, budget, and interests. They do not support a structured trip lifecycle with status transitions or history tracking.

A.3 Comparative Analysis of Existing Systems

Feature / Platform	Google Travel	TripAdvisor	Expedia	Wanderlog	AI-Assisted Travel Planner
Preference-based destination recommendation	No	Partial	No	Partial	Yes (Google Gemini API)
Manual itinerary planning (no booking required)	No	No	No	Yes	Yes
Integrated flight search	Yes	No	Yes	No	Yes
Flight wishlist / save for later	Limited	No	Limited	No	Yes
Trip lifecycle management (active → completed)	No	No	No	No	Yes
Destination browsing with weather data	Yes	Partial	No	Partial	Yes
No payment processing required	No	No	No	Yes	Yes
Single-page application, frontend-focused	No	No	No	Yes	Yes (React + Bootstrap)

A.4 Relevant Technologies and Research

A.4.1 Google Gemini API for Recommendation

In this project's implementation, the recommendation engine leverages the Google Gemini API using structured prompt engineering. This architecture allows the system to accept natural language user preferences and enforce strict formatting rules to output clear, structured JSON objects. This structured output guarantees that destination recommendations follow a predictable blueprint—including destination name, best months to visit, suggested duration, reason for recommendation, recommended hotels, and famous landmarks—which maps cleanly onto the React frontend components without requiring an intermediate server-side translation layer.

A.4.2 React and Single-Page Application Architecture

React has become a dominant framework for building Single-Page Applications (SPAs). Its component-based architecture promotes modularity and reusability, enabling independent development of discrete UI units. SPAs reduce perceived latency through client-side rendering, improving user experience for data-intensive applications. React Router extends React for client-side navigation, enabling multi-page application behaviour without server-side page reloads — a design pattern explicitly utilised in this application to navigate across the Dashboard, Planner, Trips, Flights, Destinations, and Trip History views.

A.4.3 Backend-as-a-Service and Firebase

The Backend-as-a-Service (BaaS) model abstracts server infrastructure management from frontend application development. Firebase provides authentication and a NoSQL document database (Firestore). Firestore's real-time synchronisation and document storage model allow frontend-focused applications to maintain persistence safely. For this system, user identity is managed securely via Firebase Authentication, while structured user trip states, wishlists, and active plans are stored and updated inside collections within Firebase Firestore, avoiding the necessity of a dedicated intermediate server layer.

A.4.4 RESTful API Integration in Frontend Applications

Modern travel applications depend on third-party RESTful APIs for real-time data such as flight availability, attraction information, and weather conditions. Best practices in API integration emphasize response normalization, error handling, and rate limit management. The service module pattern employed in this project (`flightService`, `destinationService`) decouples external API contracts from React component logic, normalizing external flight schemas (origin, destination, airline, price, stops, departure/arrival times) into predictable application state structures.

A.5 Research Gap

The review of existing systems and research reveals a clear gap in the available tools: no existing platform combines explicit Google Gemini API preference-based destination discovery with manual itinerary planning, integrated flight search wishlisting, and a structured trip lifecycle — within a lightweight, frontend-focused application that completely eliminates payment processing or account-linking with commercial booking providers. Commercial platforms lock planning into transactional booking flows (Expedia), while itinerary tools (Wanderlog) lack built-in flight search integrations alongside structured preference-driven engines. This project targets this workflow gap directly.

A.6 Suggested Improvements Over Existing Systems

- Structured preference input for recommendations via Google Gemini API provides transparent, immediate, and controllable personalization.
- A clear isolation between trip planning and real payment processing minimizes user friction, turning booking actions into informational wishlist links or trip attachments.
- An explicit planning state machine (Active Trips converting cleanly into Trip History) structures user records organically.

Section B: Project Planning & Management

B.1 Project Proposal Summary

The AI-Assisted Travel Planner is developed as a frontend-focused web application utilizing React, Bootstrap, Firebase Authentication, and Firebase Firestore. The scope explicitly isolates the tool as a decision-support system, deliberately excluding real financial transaction processing, payment gateways, and actual ticket fulfillment. The core interactive components deliver a complete planning framework across a Dashboard, an AI/Manual Planner page, active Trip management cards, a Flight Search module with wishlist features, and a historical archive for completed trips.

B.2 Project Phases and Planning Targets

The project outline sets targets across a typical implementation timeframe, breaking requirements gathering, interface design, service isolation, and verification into modular phases. Key development efforts focused heavily on client-side routing, service orchestration (flight, weather, and destination data formatting), and prompt optimization for structured Google Gemini API interactions.

B.3 Target Risk Matrix

The following planning matrix details identified operational challenges and recommended mitigation strategies targeted during system specification:

Risk ID	Risk Description	Target Mitigation Strategy
R-01	External flight API rate limits or schema changes breaking frontend parsing.	Abstract API requests inside modular services with fallbacks; cache structures; capture parsing faults gracefully in the UI.
R-02	Google Gemini API text outputs failing structured parsing constraints.	Enforce exact structural formatting instructions inside system instructions; write schema defensive parsing functions inside the app layer.
R-03	Scope expansion towards transactional booking infrastructure.	Maintain explicit architectural boundaries; clarify booking text as non-transactional itinerary references.

B.4 Intended Key Performance Indicators (KPIs)

System goals established during project formulation emphasize specific targets to serve as benchmarks for evaluating engineering completeness:

- **Functional Compliance:** Verification of application paths (Dashboard, Planner, Trips, Flights, Destinations, History).
- **Data Isolation:** Verification that all flight saving actions act correctly as wishlists stored dynamically in Firestore.
- **Interface Adaptability:** Rendering accuracy across typical mobile, tablet, and desktop viewports using Bootstrap grid rules.

B.5 Engineering Scope and Implementation Strategy

As a core engineering execution, development concentrated heavily on modular separation of duties within the frontend architecture. System routines are decoupled into standalone views and stateless service modules (e.g., flight logic parsing, custom preference template builders for the Google Gemini API, and dynamic reactive authentication updates via Firebase web SDK references).

B.6 Recommended Repository and Integration Practices

To promote code health and align with software engineering guidelines, the following practices are established as target approaches for version management and long-term repository maintenance:

B.6.1 Version Control Recommendations

Source code management should follow standard branch isolation practices (such as main and development branch separations). Feature-driven isolated changes (e.g., isolating flight search layout or Gemini model connector modules) are best developed in temporary context-specific feature branches before being aggregated into steady baselines.

B.6.2 Recommended Commit Conventions

Using semantic structures for commit messaging provides highly auditable code transitions. Recommended patterns follow clear prefixes:

- `feat`: when adding new system capabilities like the Manual Planner modal.
- `fix`: when debugging user interface or model output parsing errors.
- `docs`: when modifying structural inline code definitions or setup steps.

B.6.3 Verification and Quality Assurance Approach

Validation of system features is addressed through structured manual testing scripts and verification scenarios. Testing focuses on verification of user authentication persistence, accurate state mapping when transferring an AI-suggested or manually configured plan into the Active Trips collection, and error-handling routines when external API connections encounter rate boundaries or timeout conditions.